

MindSpore Transformers 大模型系列课程

MindSpore Transformers LLM预训练与微调介绍与实践

主讲人：黄生帅

昇思MindSpore研发工程师

MindSpore Transformers SIG Committer

专题一：套件介绍

《MindSpore Transformers训推Mcore架构介绍》

《MindSpore Transformers环境安装与镜像制作》

专题二：LLM训练

《MindSpore Transformers LLM预训练与微调介绍与实践》

《MindSpore Transformers LLM数据预处理介绍与实践》

《MindSpore Transformers Safetensors权重详解》

《MindSpore Transformers断点续训介绍与实践》

《MindSpore Transformers训练在线监控介绍与实践》

《MindSpore Transformers训练高可用特性介绍》

专题三：LLM推理

《MindSpore Transformers推理介绍与实践》

《MindSpore Transformers & vLLM部署与评测》

专题四：高阶开发

《MindSpore Transformers LLM推理迁移与实践》

《MindSpore Transformers & vLLM推理精度比对》

《MindSpore Transformers LLM训练迁移与实践》

《MindSpore Transformers & Megatron-LM训练精度比对》

《MindSpore Transformers训练性能调优》

《MindSpore Transformers推理性能调优》

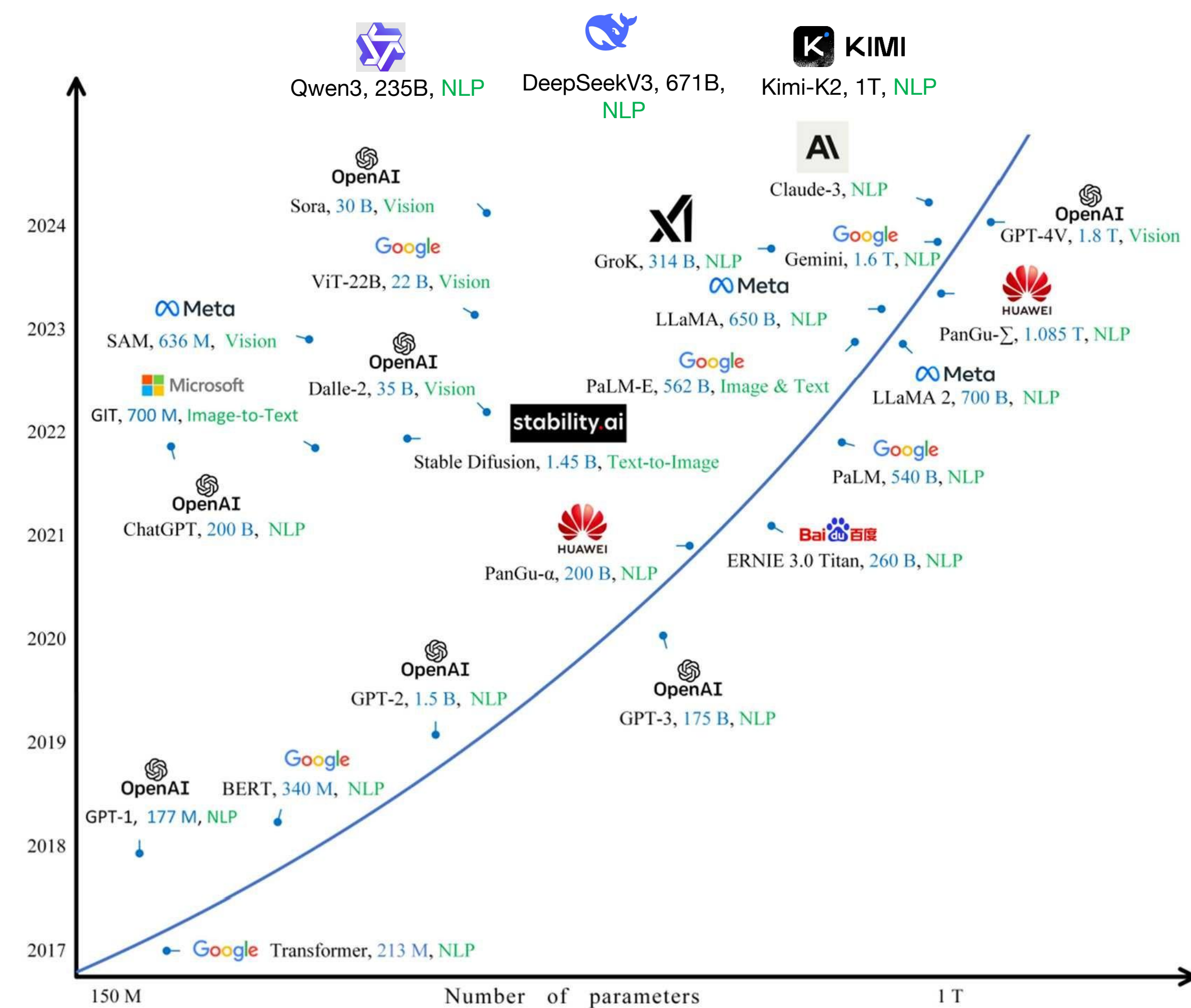
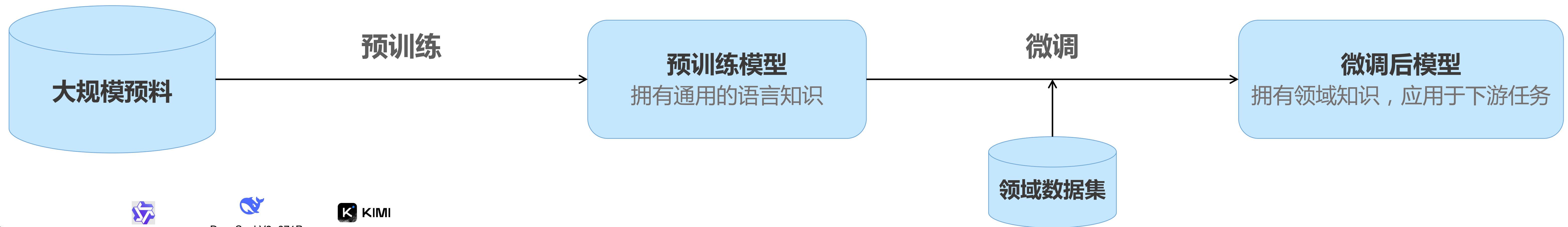
《MindSpore Transformers DeepSeek-R1蒸馏实践》

目录

1. LLM预训练与微调原理介绍
2. MindSpore Transformers训练全流程能力一览
3. MindSpore Transformers训练任务快速上手
4. 总结

1. LLM预训练与微调原理介绍

LLM预训练与微调原理介绍



LLM预训练任务：主流架构为Transformers架构，通过在大规模语料上进行学习，学习通用语言知识

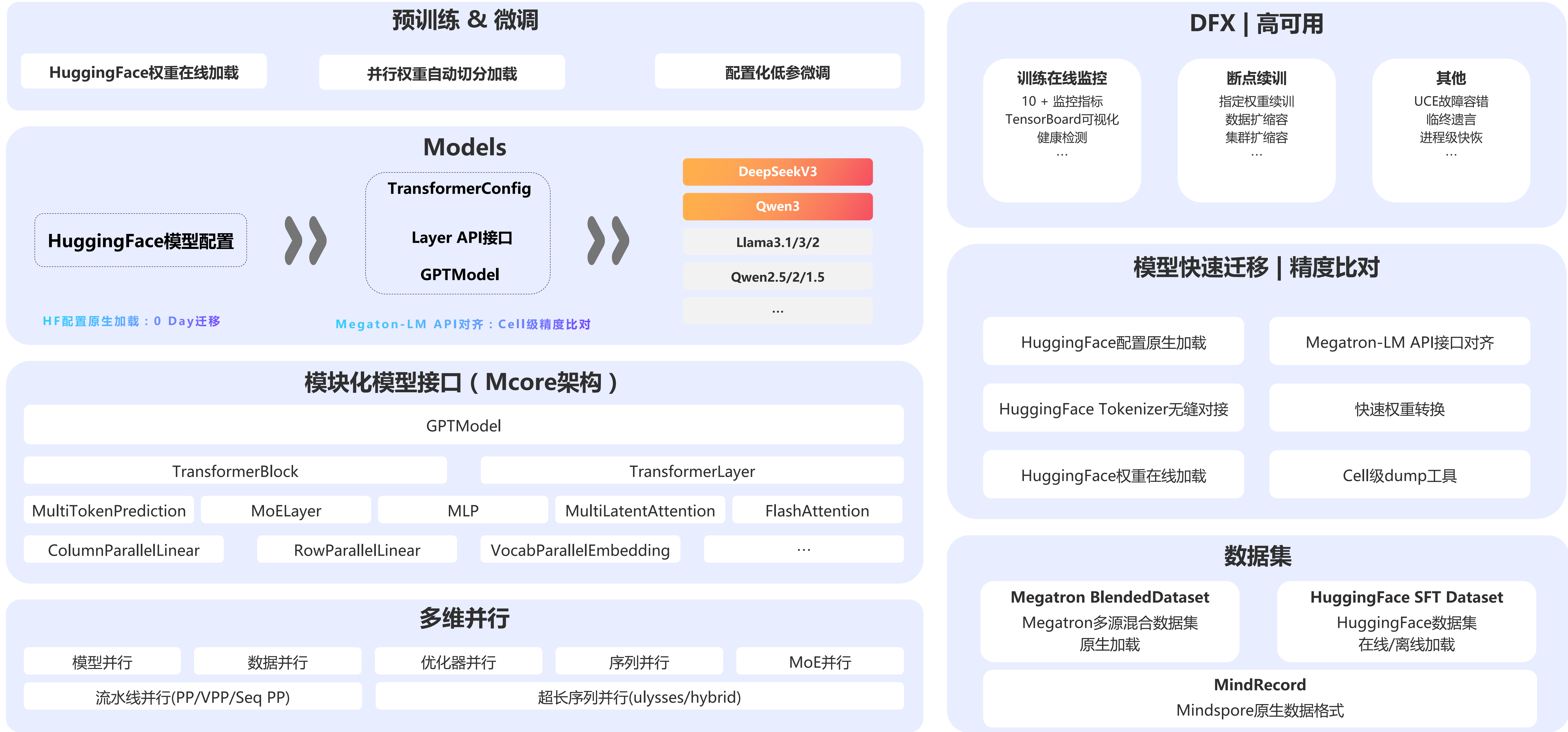
- 预训练模型通常参数规模量超大，并且逐年增大
- 开源的预训练模型通常开源在HuggingFace社区
- **挑战：**超大规模下如何实现训练？如果通过分布式并行、显存优化等手段实现高效训练？

LLM微调任务：在预训练模型基础上，针对不同任务使用不同领域数据集，融入领域知识、对齐人类偏好

- 主要方法有全参微调、低参微调、对齐技术（例如RLHF、DPO等）
- HuggingFace社区上托管了诸多领域数据集、对话数据数据，采用统一格式，可方便进行微调

Source: An overview of large AI models and their applications

2. MindSpore Transformers训练全流程能力一览



MindSpore Transformers训练全流程

任务前准备

修改训练配置

启动训练任务

训练状态监控

- 确定指定规格模型
- 数据集预处理

基本配置

数据集 | 并行配置 | 训练超参

高级配置

权重保存 | 断点续训 | 在线监控

进阶配置

健康监测 | 性能调优 | 故障快恢

msrun快速启动

单机多卡 | 多机多卡

训练状态查看

日志查看 | tensorboard

任务恢复

断点续训

任务前准备

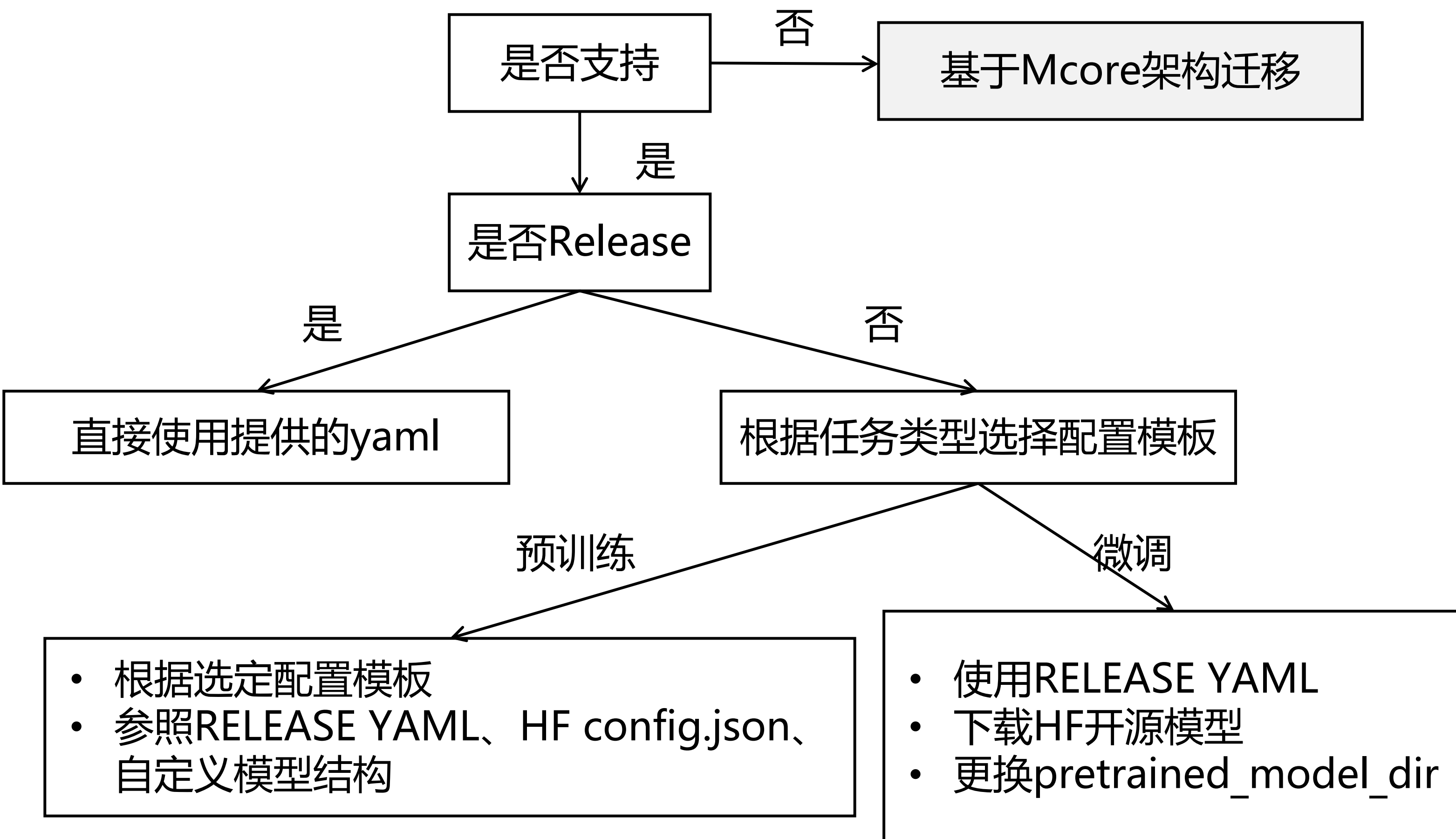
修改训练配置

启动训练任务

训练状态监控

选择模型

- **Released** (发布级)：通过测试团队验收，确定性条件下，loss 与 grad norm 精度与标杆拟合度满足标准；
- **Validated** (验证级)：通过开发团队自验证，确定性条件下，loss 与 grad norm 精度与标杆拟合度满足标准；
- **Preliminary** (初步级)：通过开发者初步自验证，功能完整可试用，训练正常收敛但精度未严格验证；
- **Untested** (未测试级)：功能可用但未经系统测试，精度和收敛性未验证，支持用户自定义开发使能；
- **Community** (社区级)：社区贡献的 MindSpore 原生模型，由社区开发维护。



```
model:
  model_config:
    vocab_size: 151936
    hidden_size: 5120
    intermediate_size: 25600
    num_hidden_layers: 64
    num_attention_heads: 64
    num_key_value_heads: 8
    params_dtype: "float32"
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
    ...
```

修改结构参数

```
model:
  model_config:
    vocab_size: 151936
    hidden_size: 5120
    intermediate_size: 17408
    num_hidden_layers: 40
    num_attention_heads: 40
    num_key_value_heads: 8
    params_dtype: "float32"
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
    ...
```

预训练任务：Qwen3-32B模型结构修改成Qwen3-14B模型结构（部分）
Key值与HF config.json一致

```
pretrained_model_dir: "/path/to/Qwen3-32B"
model:
  model_config:
    params_dtype: "float32"
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
    ...
```

修改模型文件夹

```
pretrained_model_dir: "/path/to/Qwen3-8B"
model:
  model_config:
    params_dtype: "float32"
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
    ...
```

微调任务：基于Release的微调配置，将pretrained_model_dir配置成HF文件夹
HF文件放置config.json、tokenizer.json、权重文件，支持直接复用

MindSpore Transformers训练全流程

任务前准备

修改训练配置

启动训练任务

训练状态监控

选择模型

- Released (发布级)：通过测试团队验收，确定性条件下，loss 与 grad norm 精度与标杆拟合度满足标准；
- Validated (验证级)：通过开发团队自验证，确定性条件下，loss 与 grad norm 精度与标杆拟合度满足标准；
- Preliminary (初步级)：通过开发者初步自验证，功能完整可试用，训练正常收敛但精度未严格验证；
- Untested (未测试级)：功能可用但未经系统测试，精度和收敛性未验证，支持用户自定义开发使能；
- Community (社区级)：社区贡献的 MindSpore 原生模型，由社区开发维护。

```
def get_pet_model(base_model: PreTrainedModel,
                  config: Union[dict, PetConfig]):
    pet_type = config.get("pet_type")

    if not PET_TYPE_TO_MODEL_MAPPING.get(pet_type):
        logger.warning("%s doesn't have pet model currently.",
                        pet_type)
        return base_model
    ...
    # return pet model.
    return PetModel(config=config, base_model=base_model)
```

Pretrained weights



⊕

A

B

MindSpore Transformers低参微调实现原理

```
pretrained_model_dir: "/path/to/Qwen3-32B"
model:
  model_config:
    params_dtype: "float32"
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
    ...
```

```
pretrained_model_dir: "/path/to/Qwen3-8B"
model:
  model_config:
    params_dtype: "float32"
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
    ...
  pet_config:
    pet_type: lora
    lora_rank: 16
    lora_alpha: 16
    lora_dropout: 0.05
    target_modules: ".*wq|.wk|.wv"
```

MindSpore Transformers低参微调配置方法

通过pet_config指定微调算法，指定微调目标模块

是否支持

否

基于Mcore架构迁移

是

是否Release

是

直接使用提供的yaml

否

根据任务类型选择配置模板

预训练

微调

- 根据选定配置模板
- 参照RELEASE YAML、HF config.json、自定义模型结构

- 使用RELEASE YAML
- 下载HF开源模型
- 更换pretrained_model_dir

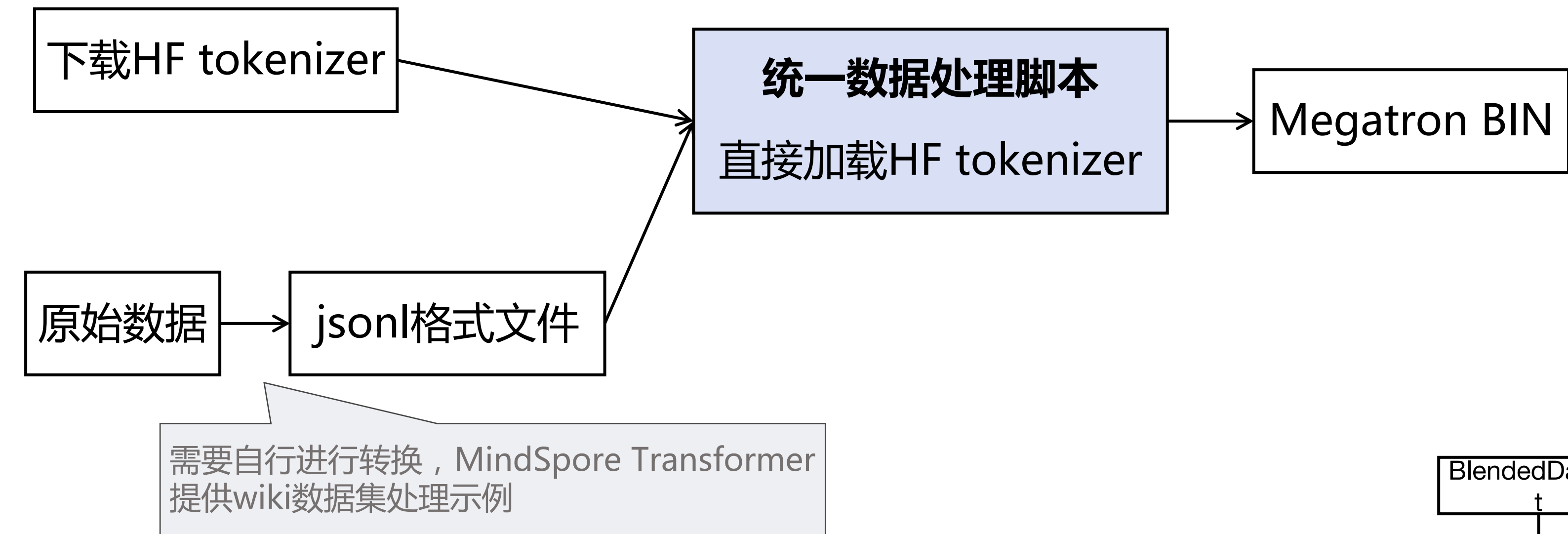
任务前准备

修改训练配置

启动训练任务

训练状态监控

数据预处理：预训练数据集

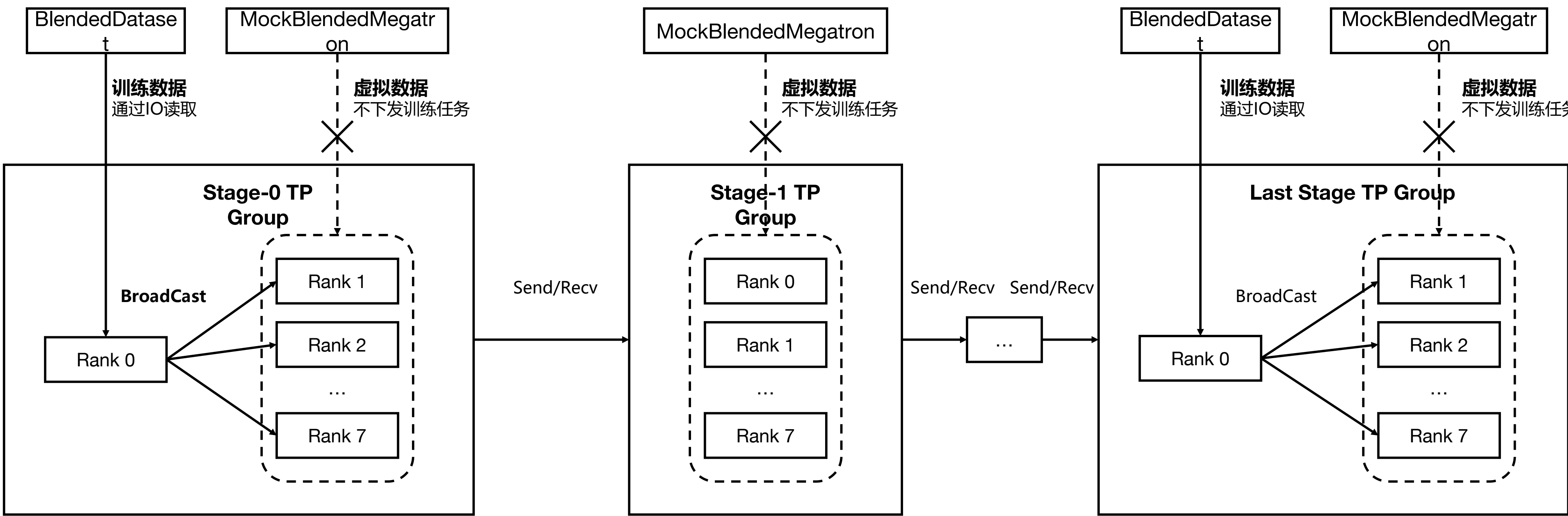


```
# jsonl数据样例，与Megatron-LM 数据一致，每一行需要包含 "text"
{"src": "www.nvidia.com", "text": "The quick brown fox", "type": "Eng", "id": "0"}
{"src": "The Internet", "text": "jumps over the lazy dog", "type": "Eng", "id": "42"}

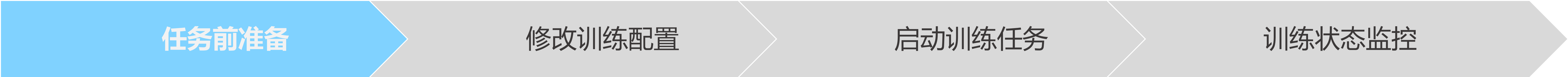
# 转换数据
python toolkit/data_preprocess/megatron/preprocess_indexed_dataset.py \
--input /path/data.jsonl \
--output-prefix /path/megatron_data \
--tokenizer-type HuggingFaceTokenizer \
--tokenizer-dir /path/to/huggingface/tokenizer
```

BlendedMegatronDatasetDataLoader：用于加载Megatron BIN

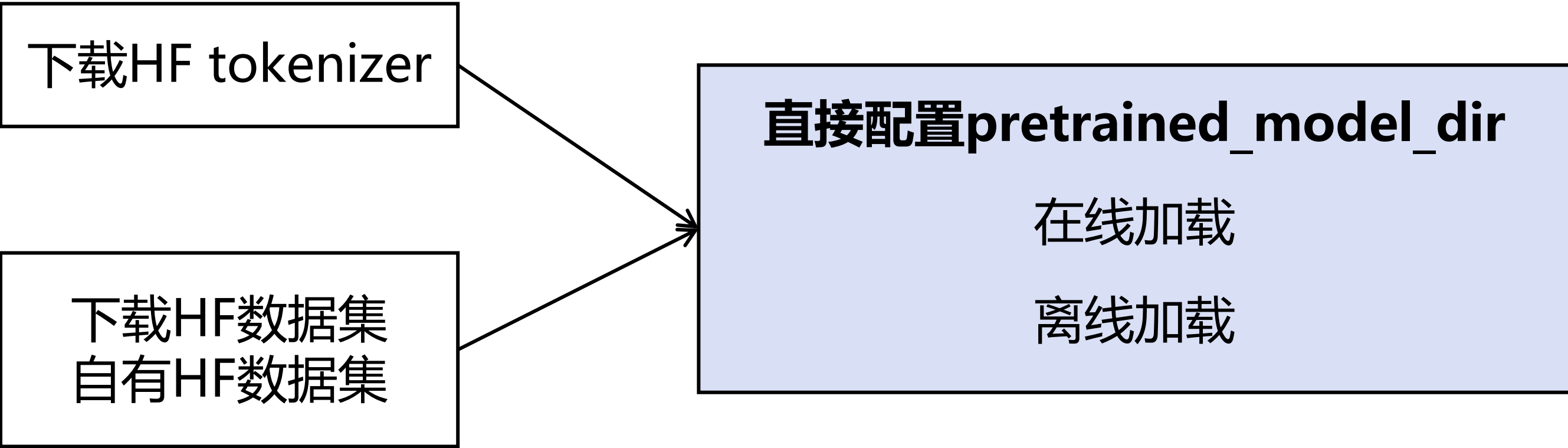
支持功能	功能说明
多源混合	支持加载多BIN文件按比例混合
数据去冗余	支持同TP/PP域数据仅一个RANK通过IO读取数据，减少IO压力
EOD压缩	支持EOD Attention压缩Mask生成



数据去冗余原理示意图



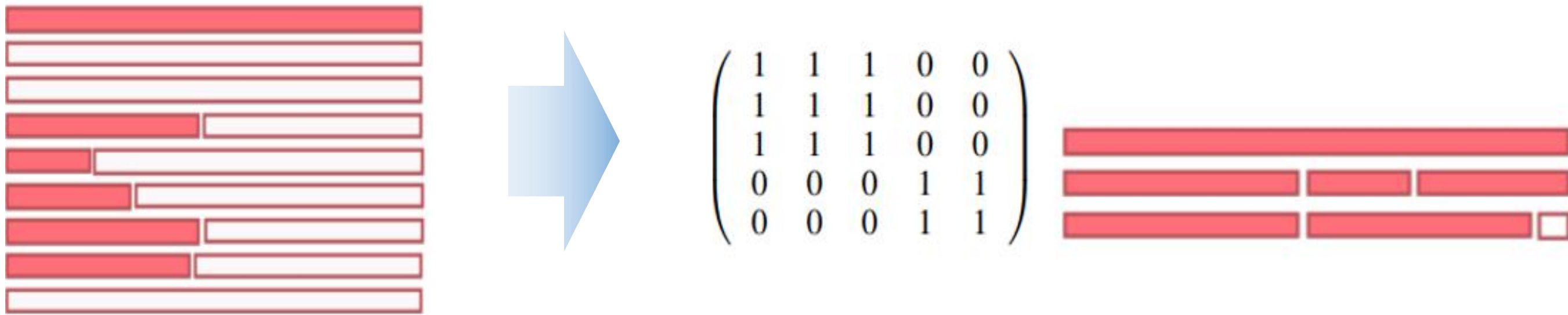
数据预处理：微调数据集



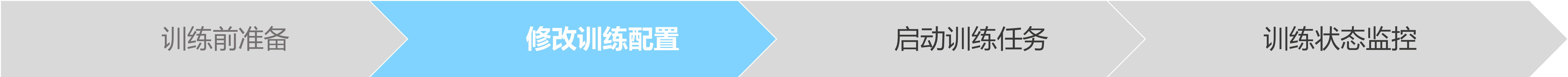
```
pretrained_model_dir: "/path/to/model/dir"      HF 模型文件夹
train_dataset: &train_dataset
data_loader:
  type: HFDataLoader
  load_func: "load_dataset"
  path: "llm-wizard/alpaca-gpt4-data-zh"        HF数据集ID
  split: "train"
  handler:
    - type: take
      n: 2000
    - type: AlpacaInstructDataHandler
      seq_length: 4096
      padding: False
      tokenizer:
        trust_remote_code: True
        padding_side: "right"
    - type: PackingHandler
      seq_length: 4096
      pack_strategy: "pack"
```

HF DataLoader：用于加载微调数据集

支持功能	功能说明
HF数据集直接加载	支持在线下载数据集加载、支持离线加载
Handler操作	支持Alpaca格式数据直接转换（内置）
	复用HF tokenizer（内置）
	数据可选列（内置）
	微调对话数据支持packing（内置）
	支持transformer datasets配置 ：rename_column, remove_columns, sort, shuffle, flatten, take等
	Handler扩展 ：支持任意数据处理自定义Handler注册
数据去冗余	支持同TP/PP域数据仅一个RANK通过IO读取数据，减少IO压力
EOD压缩	支持EOD Attention压缩Mask生成，配合packing特性实现训练效率提升



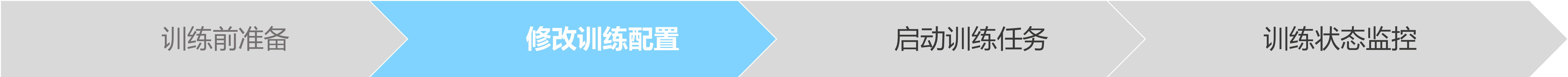
微调数据packing原理示意图



配置类型	类型说明	配置项	配置指导
基础配置	通过配置该部分配置，能够基于当前模型结构下，拉起一个简单的训练任务	数据集	数据集使用
		并行配置	并行配置项说明 并行配置指南
		训练超参	模型训练配置
高级配置	通过配置该部分配置，可支持训练任务执行后，对训练任务的训练状态进行感知，并保障多次训练任务的连贯	权重保存	Callbacks 配置 CheckPointMonitor Safetensors 权重使用指南
		断点续训	断点续训示例 Safetensors 权重使用指南
		在线监控	训练指标监控
进阶配置	通过配置该部分配置项，可支持训练过程的健康监测、故障快恢及性能调优，实现在不同集群规模下稳定并高性能训练	健康监测	数据跳过与健康监测
		性能调优	训练内存优化 性能调优指南
		故障快恢	高可用特性

Qwen3 32B预训练任务示例YAML配置

```
# 数据集配置
train_dataset: &train_dataset
data_loader:
  type: BlendedMegatronDatasetDataLoader
  datasets_type: "GPTDataset"
config:
  seq_length: 4096 # Sequence length of the dataset
  data_path: # path for the Megatron dataset
    - "1"
    - "/path/to/dataset_file"
...
# 并行配置
parallel_config:
  data_parallel: 1 # Number of data parallel
  model_parallel: 4 # Number of model parallel
  pipeline_stage: 4 # Number of pipeline parallel
  micro_batch_num: 4 # Pipeline parallel micro_batch size
...
# 训练配置
runner_config:
  epochs: 2
  batch_size: 1
  gradient_accumulation_steps: 1
# 优化器配置
optimizer:
  type: AdamW
  betas: [0.9, 0.95]
  eps: 1.e-8
  weight_decay: 0.0
# 学习率配置
lr_schedule:
  type: ConstantWarmUpLR
  learning_rate: 1.e-6
  warmup_ratio: 0
  total_steps: -1
```

重计算 & 并行配置

为了在有限的集群规模下，完成超大规模模型的高效训练；往往会通过对模型进行切分，MindSpore Transformers提供了7中并行方法；性能调优时，配合流水线并行调度，重计算等设置，实现高性能训练。

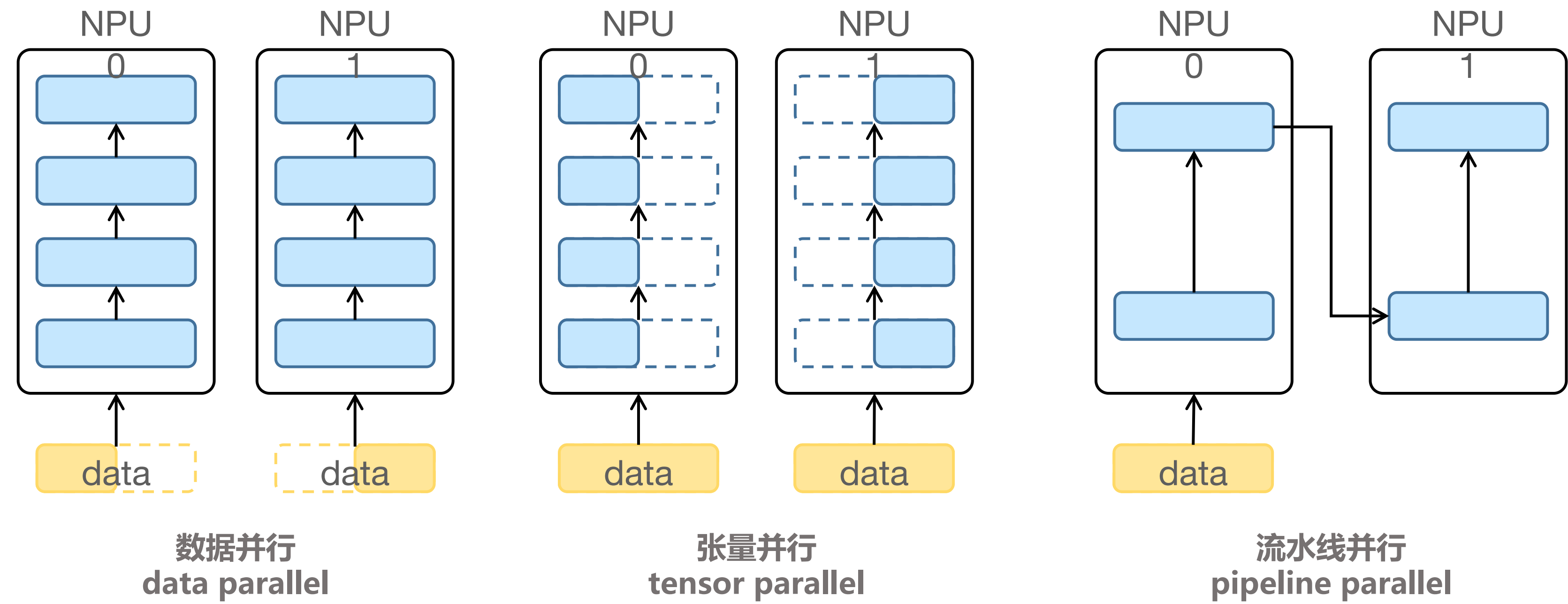
```
# 并行配置
parallel_config:
  data_parallel: 1      # 数据并行
  model_parallel: 4     # 张量并行
  pipeline_stage: 4     # 流水线并行
  micro_batch_num: 4
  expert_parallel: 4    # 专家并行
  context_parallel: 2   # 长序列并行

pipeline_config:
  pipeline_interleave: True
  pipeline_scheduler: "zero_bubble_v"
```

```
# 完全重计算
recompute_config:
  recompute: True

# 选择重计算
recompute_config:
  recompute: [4, 1, 0, 0]
  select_recompute:
    'feed_forward.mul': [11, 13, 13, 11]
  comm_recompute: True
  recompute_slice_activatio: True
```

并行策略	特点
数据并行（DP）	此方法性能最优，但内存不会减少
模型并行（TP）	此方法最省内存，但通信量很大
流水线并行（PP）	此方法通信量较小，但会存在计算闲置（bubble），需要调整负载均衡及配置重计算策略
优化器并行（OP）	此方法可以明显节省内存，通信量较小
序列并行（SP）	减少内存与Norm的部分计算量
专家并行（EP）	MoE架构使用，能够节省专家参数内存，引入很大通信量
长序列并行（CP）	能够有效减少激活内存，引入很大通信量



训练前准备

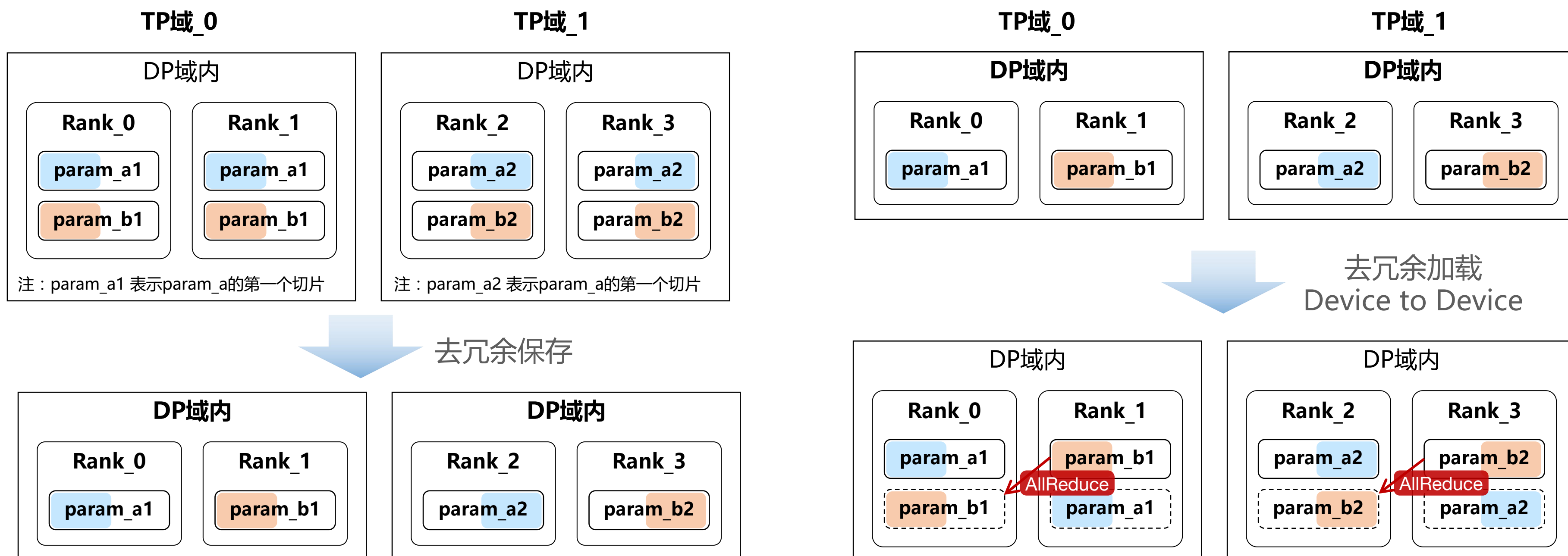
修改训练配置

启动训练任务

训练状态监控

权重相关配置

训练保存 & 加载：支持safetensors格式的保存与加载，另外支持去冗余功能



```
# 权重保存
callbacks:
  - type: CheckpointMonitor
    prefix: "qwen3"
    save_checkpoint_steps: 50
    keep_checkpoint_max: 1
    checkpoint_format: "safetensors"

# 权重加载
load_checkpoint: /path/to/safetensors/dir
load_ckpt_format: "safetensors"
remove_redundancy: True
```

断点续训: 在模型训练过程中，可能因为各种因素导致训练中断，需要重新拉起训练并且恢复到上一次训练中断时的状态继续训练，如右配置给了一个在共享存储场景下，配置权重文件夹，从最后一个保存完整且step一致的权重进行续训

```
# 断点续训: 基于最后一个保存完整且step一致的权重进行续训
load_checkpoint: /path/to/resume/safetensors/dir
resume_training: True
load_ckpt_format: "safetensors"
```


任务前准备

修改训练配置

启动训练任务

训练状态监控

run_mindformer.py

MindSpore Transformers提供了一键式拉起预训练、微调的任务编排脚本run_mindformer.py，可以通过执行该脚本启动对应配置下的训练任务；

```
# 使用run_mindformer.py 启动一个单卡任务
run_mindformer.py --config {CONFIG_PATH} --run_mode {train/finetune} --use_parallel False
```

msrun启动

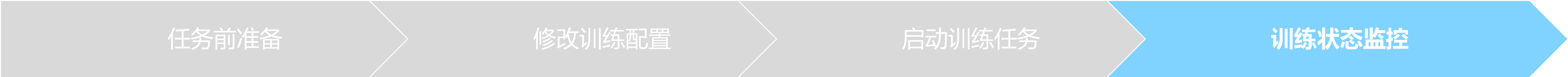
MindSpore提供的动态组网启动方式的封装，用户可使用msrun以单个命令行指令的方式在各节点拉起多进程分布式任务，并且无需手动设置动态组网环境变量，MindSpore Transformers提供了更便捷的任务封装脚本msrun_launcher.sh，供用户用于不同场景下的快速启动。

```
# 默认启动一个单机8卡任务
bash scripts/msrun_launcher.sh "run_mindformer.py
--config path/to/xxx.yaml \
--run_mode {train/finetune} \
--use_parallel True"
```

单机任务拉起方式

```
# 节点0, 节点ip为192.168.1.1, 作为主节点, 总共16卡且每个节点8卡
bash scripts/msrun_launcher.sh "run_mindformer.py --config {CONFIG_PATH} --run_mode {train/finetune} --
use_parallel True" 16 8 192.168.1.1 8118 0 output/msrun_log False 300
# 节点1, 节点ip为192.168.1.2, 节点0与节点1启动命令仅参数NODE_RANK不同
bash scripts/msrun_launcher.sh "run_mindformer.py --config {CONFIG_PATH} --run_mode {train/finetune} --use_parallel
True" 16 8 192.168.1.1 8118 1 output/msrun_log False 300
```

多机任务拉起方式



日志信息

```
●●●
# 关键字 Network Parameters 打印当前运行模型的可训练参数量
2025-06-28 07:35:35,778 - xxxx - INFO - Network Parameters: 8030 M.
# 进入训练迭代过程, 会根据设定的epoch数以及数据集大小打印对应step的训练输出值
2025-06-28 07:39:23,510 - xxxx - INFO - { Epoch:[ 1/ 2], step:[ 2/ 6500], loss: 11.905, per_step_time: 2775ms, lr: 2.5641025e-08, overflow cond: False, loss_scale: 1.0, global_norm: [45.702465], train_throughput_per_npu: 171.176T
# 通过在CheckpointMonitor设定对应保存step, 日志中会打印进入权重保存过程
2025-06-28 07:38:49,509 - xxxx - INFO - .....Saving ckpt.....
2025-06-28 07:38:49,514 - xxxx - INFO - global_batch_size: 8
2025-06-28 07:38:49,518 - xxxx - INFO - epoch_num: 1
2025-06-28 07:38:49,522 - xxxx - INFO - step_num: 10
2025-06-28 07:38:49,526 - xxxx - INFO - global_step: 10
2025-06-28 07:39:20,730 - xxxx - INFO - Finish saving ckpt of epoch 1 step 10
```

打印值	数值说明
Epoch:[1/ 2]	当前epoch/设定的epoch
step:[2/ 6500]	当前epoch下step/当前epoch总step数
loss: 11.905	表示当前step的损失输出值
per_step_time: 2775ms	表示当前step耗时时间，时间包含数据获取、前向计算、反向计算
lr: 2.5641025e-08	当前step学习率
overflow cond: False	当前step算子计算是否存在溢出，False为无溢出
loss_scale: 1.0	当前step的损失放缩值
global_norm: [45.702465]	当前step的所有参数的norm值
train_throughput_per_npu: 171.176T	当前step 每张NPU实际计算消耗的算力值，可使用该值与机器的实际算力进行计算模型训练MFU

任务前准备

修改训练配置

启动训练任务

训练状态监控

可视化在线监控

通过训练任务Yaml配置训练数据在线监控，监控指标涵盖Loss/学习率/全局梯度范数/执行卡梯度范数/数据吞吐/算力吞吐/优化器状态/权重状态等10+指标

训练指标监控: 多种监控方式/多种监控指标

monitor_config:

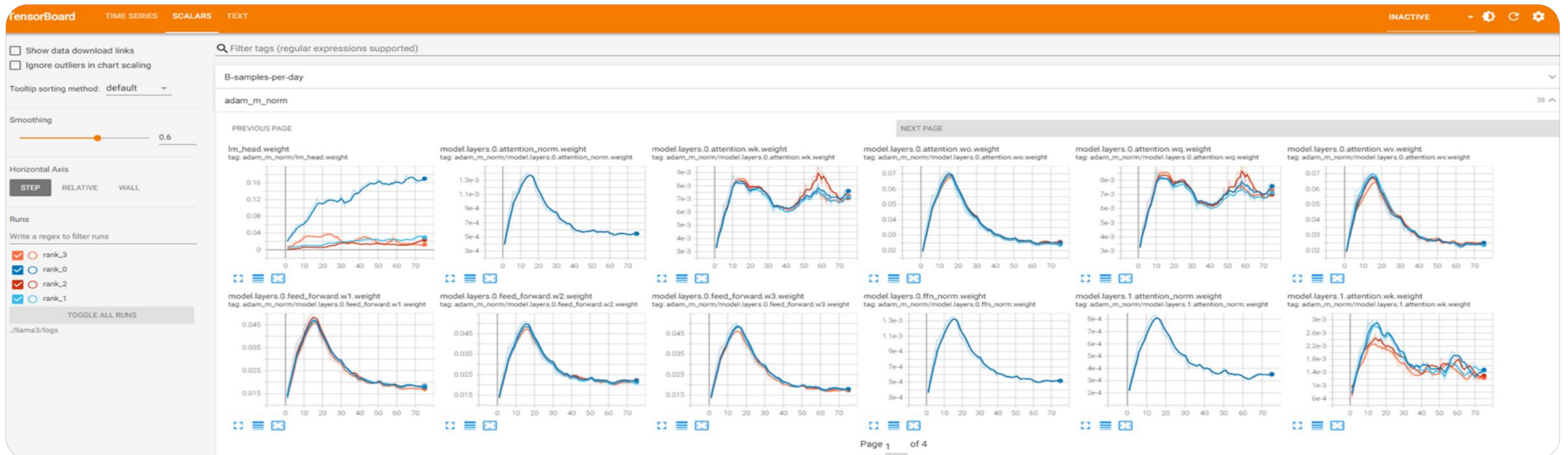
monitor_on: True

local_loss_format: [“log”, “tensorboard”]

local_norm_format: [“log”, “tensorboard”]

optimizer_state_format: [“tensorboard”]

weight_state_format: [“tensorboard”]



3. MindSpore Transformers训练任务快速上手

实操演示

MindSpore Transformers Qwen3预训练及微调

□ MindSpore Transformers全流程能力

MindSpore Transformers 训练包含4个流程：训练前准备、修改训练配置、启动训练任务，训练状态监控。

MindSpore Transformers训练前准备：通过MindSpore Transformers的模型支持度及任务类型，选定好对应模型规格，并将数据处理为Megatron-BIN或微调数据集

MindSpore Transformers修改训练配置：MindSpore Transformers提供基础、高级、进阶三种不同等级的配置项，用于执行不同级别的任务；

MindSpore Transformers启动训练任务：通过msrun + run_mindformer.py可以方便的启动单机、多机任务；

MindSpore Transformers训练状态监控：MindSpore Transformer提供了多种日志信息以及Tensorboard对训练状态进行监控；

SIG组织

昇思社区

昇腾社区

魔乐社区

SIG > MindSpore Transformers

MindSpore Transformers

https://gitee.com/mindspore/community/tree/master/signs/mindspore_transformers

MindSpore Transformers SIG (Special Interest Group) 是 MindSpore 开源社区下聚焦 Transformer 类大模型全流程开发与优化的技术团队，涵盖大模型的预训练、微调、推理与部署等关键环节。SIG 团队致力于提升 Transformer 类大模型在性能与易用性方面的表现，推动其在实际场景中的高效应用与落地。

核心成员(6)

Maintainers (2)

L

Lin-Bert

He Qinglin

S

suhaibo

Su Haibo

Committers (4)

C

chenrayray

Chen Xinrui

H

hss-shuai

Huang Shengshuai

R

renyujin

Ren Yujin

Z

zyw-hw

zhang Youwen

会议与活动

工作会议

查看会议纪要 >

会议时间：每月一次，北京时间周四 16:00 - 17:00 以公开线上会议形式讨论SIG组议题。

会议前3-5天会在邮件列表中发起sig组相关议题收集，并发出会议时间和会议链接。

邮件

dev@mindspore.cn

订阅邮件

最新日程：2025/09/04

2025/09

04

2025/08

07

2025/07

03

MindSpore Transformers SIG月例会

2025-09-04 16:00 - 17:00

会议详情：-

发起人：chenrayray

会议时间：16:00-17:00

会议平台：tencent

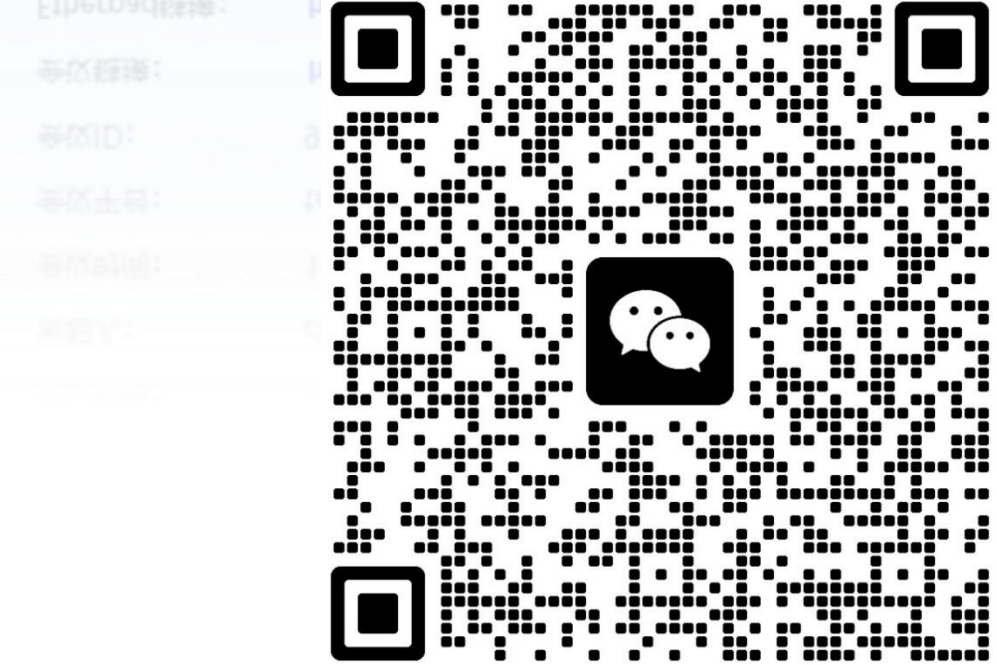
会议ID：952843996

会议链接：<https://meeting.tencent.com/dm/Fig5yhhFz7E0>

Etherpad链接：<https://etherpad.mindspore.cn/p/sig-MindSpore-Transformers-meetings>



课程交流群



MindSpore Transformers SIG群



MindSpore Transformers 内测文档



MindSpore Transformers 开源仓库

谢谢观看！

LLM预训练与微调介绍与实践
MindSpore Transformers 大模型系列课程